# 6.1 JAVASCRIPT Y LA PROGRAMACIÓN ORIENTADA A OBJETOS

## 6.1.1 ¿QUÉ ES LA POO?

Las siglas **POO** se corresponden a **Programación Orientada a Objetos,** un término que ya tiene un largo recorrido dentro de la ciencia de programación de computadoras y que cobró una enorme dimensión a finales de los 80 hasta su total implantación en la década de los 90 del siglo pasado.

En definitiva, la POO es un modelo de programación. Su éxito se debe a que ha conseguido facilitar enormemente la creación de aplicaciones al conseguir que una aplicación sea un conjunto de objetos que se comunican. Cada objeto se programa de forma independiente, consiguiendo una modularización mayor que la propia programación modular. Los objetos son una estructura que aglutina datos y acciones (funciones).

La idea es que los objetos envían y reciben mensajes entre sí. Un objeto será un elemento de la aplicación capaz de recibir mensajes que provocan un procesamiento de dicho de mensaje hasta producir un resultado que, a su vez, determinará un envío de mensajes a otro objeto.

El mantenimiento de las aplicaciones es más sencillo ya que nos concentramos en hacer que cada objeto funcione debidamente, y, cuando dicho objeto funciona, lo podemos utilizar en todas las aplicaciones que deseemos.

La idea de objeto es la de una estructura que aglutina datos y acciones. Los datos son los atributos o propiedades de los objetos. Si por ejemplo, un objeto representara un ***coche,*** los datos (que se llaman **atributos** o **propiedades)** podrían ser la marca, el color, la potencia, etc. Las acciones (que se llaman **métodos)** podrían ser arrancar, parar, acelerar, repostar... es decir, lo que un coche es capaz de hacer.

Una vez que un tipo de objeto (en terminología POO un tipo de objeto se conoce como **clase)** se ha definido, se puede utilizar las veces que queramos y ya no nos preocuparemos por él. Programar aplicaciones consistirá en analizar y diseñar los objetos que necesitamos y cómo se han de comportar y comunicar entre sí.

## 6.1.2 CARACTERÍSTICAS DE LA POO

Hay cierto debate sobre si JavaScript es realmente un lenguaje orientado a objetos. Ese debate se debe a que se considera que otros lenguajes (como C++, java o Python) implementan de forma más completa el paradigma de la POO.

La POO se dice que ha de tener estas características:

- **Abstracción.** Propiedad que permite reutilizar un objeto sin conocer su funcionamiento interno, nos bastará con que se nos proporcione lo que se conoce como su **interfaz externa.** La interfaz son las propiedades y métodos que permiten manipular y utilizar el

objeto. En definitiva, la abstracción en la POO nos permite reutilizar el código de forma óptima, al poder utilizar objetos sin conocer los detalles de su implementación.

- **Encapsulamiento.** Capacidad de que los objetos puedan ocultar algunos métodos y propiedades de modo que, podamos elegir solo los que nos interesan que queden visibles.

- **Polimorfismo.** Esta característica permite conseguir que diferentes tipos de objetos puedan tener métodos con el mismo nombre, pero que actuarán de forma diferente. Por ejemplo, los objetos de clase círculo podrían tener un método llamado suma que permite unir su superficie a la de otro círculo, pero la clase **númeroGigante** podría tener el mismo método, pero para hacer sumas aritméticas.

- **Herencia.** Los diferentes tipos de objetos se suelen relacionar con **clases** de objetos. La mayoría de lenguajes define clases y luego crea objetos a partir de esas clases, es decir: objetos que son de una determinada clase. La herencia permite que haya clases que hereden las características de otras clases. Por ejemplo, la clase ***Automóvil*** puede tener un método que se llame ***acelerar*** y otro que se llame ***frenar.*** Si definimos una clase llamada ***Coche,*** indicando que esa clase deriva de la clase ***Automóvil,*** los coches también podrán frenar y acelerar.

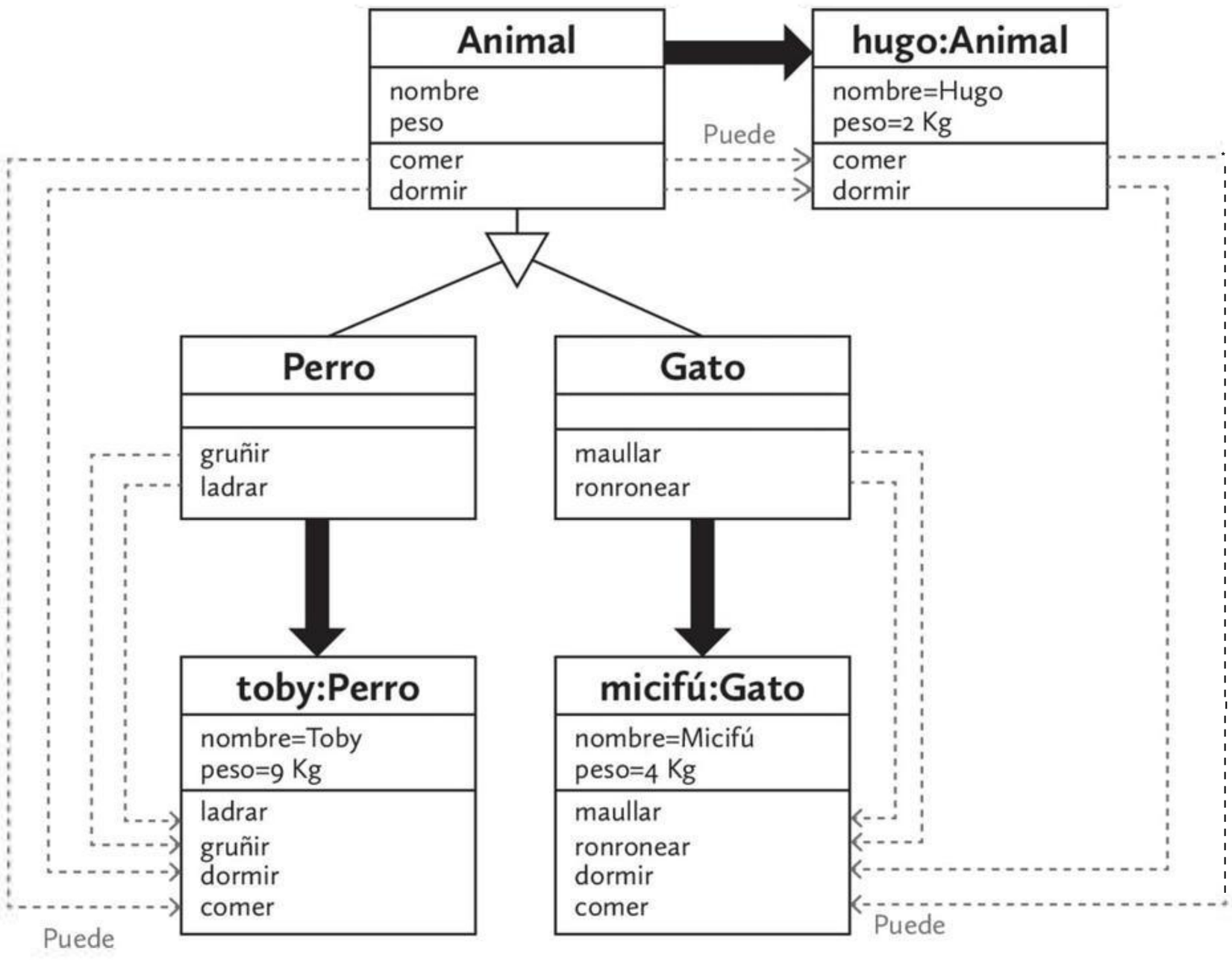



**Figura 6.1:** Ejemplo de uso de clases y objetos con herencia en la POO

En la Figura 4.1, **Animal, Perro** y **Gato** son clases de objetos. Mientras que **toby, hugo** y **micifú** son objetos ya concretos. Perro y Gato son clases que descienden de la clase **Animal** (son animales). El objeto ***toby,*** que es un perro, hace lo que hacen los perros (ladrar y gruñir), pero también

heredan lo que hace cualquier animal (comer y dormir). Lo mismo ocurre con ***micifú,*** hace cosas de gatos y también las cosas que comparte con cualquier otro animal. De ***hugo*** solo sabemos que es un animal genérico por ello solo puede hacer las cosas que hace cualquier otro animal.

### 6.1.3 JAVASCRIPT COMO LENGUAJE ORIENTADO A OBJETOS

Hay lenguajes que llevan a rajatabla las características vistas en los apartados anteriores, otros aportan cierta flexibilidad, o bien una reinterpretación de las mismas. Es todo un debate, a veces polémico, si JavaScript es un lenguaje realmente orientado a objetos o no.

Sin entrar a fondo en este debate, cierto es que JavaScript trabaja de forma diferente los objetos con respecto a lenguajes como java o C++, pero indudablemente los objetos son importantísimos en JavaScript e independientemente de cómo implementa el paradigma, toda aplicación JavaScript utiliza objetos y permite modular el código basándonos en interacciones entre objetos.

Sin duda, JavaScript es un lenguaje orientado a objetos basado en prototipos. A partir de ES6 incluye la palabra clave **class**, que permite definir clases y herencia con una sintaxis más clara y similar a la de otros lenguajes. Sin embargo, internamente, las clases siguen basándose en el mecanismo de prototipos.

## 6.2 USO DE OBJETOS

### 6.2.1 ACCESO A PROPIEDADES Y MÉTODOS

Si tenemos un determinado objeto, el acceso a sus propiedades y métodos se consigue gracias al operador punto (.) Para acceder a una propiedad de un objeto, la sintaxis es:

```
Objeto.propiedad
```

Cambiar el valor de una propiedad es posible, por ejemplo, así:

```
coche.co1or="Rojo";
```

Se ha modificado la propiedad ***color*** del coche. También podemos acceder usando esta forma:

```
objeto["propiedad"]
```

Por ejemplo:

```
coche["color"]=”Rojo”;
```

Para utilizar métodos, la sintaxis es:

```
objeto.metodo([parámetros])
```

Ejemplo:

```
coche.acelerar(25);
```

También es posible con:

```
coche["acelerar"](25);
```

## 6.2.2 OBJETOS LITERALES

Los objetos más sencillos que podemos crear en JavaScript son los llamados **literales** o **instancias directas.** Son objetos en los que se pueden definir directamente sus propiedades y métodos sobre la marcha. Por ejemplo:

```
let punto=new 0bject(); //punto es un objeto, por ahora vacío
punto.x=5; //Definimos la propiedad x y le damos valor
punto.y=punto.x*2;
//Definimos la propiedad "y" haciendo que valga el
// doble de lo que vale "x", es decir: 10
punto.mostrarCoordenadas=function(){
    return `(${punto.x},${punto.y})`;
}
console.log(punto.mostrarCoordenadas( )) ; //Escribe (5,10)
```

La primera línea del código anterior:

```
let punto=new 0bject();
```

Indica que la variable llamada ***punto*** es un objeto. Hay una instrucción más sencilla y equivalente a la anterior:

```
let punto={};
```

Las llaves permiten indicar que la variable punto es un objeto vacío. Tras esa instrucción podemos definir propiedades y métodos de la misma forma. Otra posibilidad es crear el objeto y directamente asignarle propiedades y métodos:

```
let punto={
        x: 19,
        y:36,
        mostrarCoordenadas:function(){
                return `(${punto.x},${punto.y})`;
        }
};
console.log(punto.mostrarCoordenadas());
```

Esta forma de definir objetos se basa en la forma:

```
{
propiedad1:valor1,
propiedad2:valor2,
…
método1:function(…){…},
…
}
```

El nombre de las propiedades y métodos puede ir entrecomillado, por lo que se admiten incluso nombres de propiedades y métodos con guiones y espacios en blanco (aunque no es muy recomendable hacerlo). El orden de las propiedades y métodos no es estricto, podemos empezar por métodos, luego definir propiedades, luego definir más métodos, etc. Es decir, el orden lo decidimos a voluntad. Ahora bien, es recomendable, para una mejor legibilidad del código, definir primero las propiedades y luego los métodos.

```
let libro = {
    título : "Manual de UFOlogía",
    " n-serie"      :   "18702",     //Las propiedades pueden llevar espacios,
                //guiones, etc. por lo que en esos casos se
                //usan comillas
    autores : [ "Pedro Martínez","Ana León"],
                //Esta propiedad es un array
    editorial : { //La editorial es otro objeto
        nombre : "Inexistente S.A.",
        país : "España"
    }
};
console.log(1ibro.título); //Escribe: Manual de UFOLogía
console.log(libro["n-serie"]); //Escribe: 187C2
console.log (1ibro.editorial.nombre); //Escribe Inexistente S.A.
```

Los objetos pueden ser todo lo complicados que queramos, una propiedad de un objeto puede ser un array, un mapa un conjunto, otro objeto, etc.

## 6.2.3 OBJETO **THIS**

En alguno de los códigos de ejemplo anteriores ya hemos utilizado la palabra reservada **this.** Para entender mejor su uso, veamos el siguiente ejemplo de código:

```
let punto={
        x: 19,
        y: 36,
        mostrarCoordenadas:function(){
                return `(${x},${y})`;
        }
};
console.log(punto.mostrarCoordenadas());
```

El resultado de ese código provoca un error de variable indefinida. La razón es que no hay una variable ***x,*** lo que hay es una propiedad ***x*** (lo mismo pasa con y). Por lo tanto, desde la función que se asigna a **mostrarCoordenadas** no se puede acceder a ninguna variable llamada ***x.***

```
let punto={
        x:19,
        y: 36,
        mostrarCoordenadas:function(){
```

```
            return `(${punto.x},${punto.y})`;
        }
};
console.log(punto.mostrarCoordenadas());
```

Al hacer referencia a ***punto.x y punto.y,*** ya no hay indefinición, está claro que ahora, sí sabemos que nos referimos a las propiedades del punto.

Pero veamos, finalmente este código:

```
let punto={
        x: 19,
        y: 36,
        mostrarCoordenadas:function(){
            return `(${this.x},${this.y})`;
        }
};
console.log(punto.mostrarCoordenadas());
```

El resultado es el mismo. ¿A quién hace referencia **this?** Al objeto actual, que en este caso es accesible a través de la variable ***punto,*** pero que con código más complejo no sería tan fácil llegar al objeto. **This,** nos permite abstraemos y llegar al objeto propietario de la propiedad o el método que estamos definiendo, sea el que sea.

Para entender mejor las posibilidades de this veamos este código que es un poco más complejo:

```
function doblarX(){
        this.x*=2;
}
let punto={
        x: 15,
        y:7,
        doble:doblarX
};
let incognita={
        x: 5,
        dbl:doblarX
};
punto.doble();
incognita.dbl();
console.log(punto.x) //Escribe 30, doble de 15
console.log(incognita.x) //Escribe 10, doble 5
```

La función ***doblarX*** sirve para multiplicar por dos la propiedad ***x*** del objeto que sea propietario de esta propiedad. Cuando esta función se asigna al método ***doble*** del objeto ***punto*** lo que ocurrirá, cuando se la invoque, es que se doblará el valor ***x*** de la propiedad del punto. Lo mismo ocurre cuando asignamos la función ***doblarX*** al método ***dbl*** del objeto ***incógnita.*** Lo interesante es que la palabra this permite llegar al objeto en cuestión y por eso podemos definir métodos, propiedades y funciones de manera más abstracta y que funcionan incluso en objetos distintos.

# 6.3 USO AVANZADO DE OBJETOS

## 6.3.1 CREAR OBJETOS A TRAVÉS DE CONSTRUCTORES

En JavaScript existe una forma de crear objetos que resulta familiar para los programadores acostumbrados a lenguajes orientados a clases.

Se trata de utilizar **constructores**, que son métodos especiales dentro de una **clase** y sirven para inicializar los objetos con determinados valores al momento de crearlos.

El operador new permite generar nuevos objetos a partir de una clase. A este proceso se le conoce como **instanciación** u **obtención de instancias**, es decir, la creación de ejemplares concretos de una clase con sus propios datos y propiedades.

Vamos a crear una clase:

```
class Person {
  constructor(firstname, lastname, birthday) {
    this.firstname = firstname;
    this.lastname = lastname;
    this.birthday = new Date (birthday);
  }
  getAge() {
    const today = new Date();
    let age = today.getFullYear() - this.birthday.getFullYear();
    return age;
  }
}
const p = new Person('Paul', 'Almunia', '1966-08-07');
console.log(p.getAge()); // calcula la edad real
```

A diferencia de las funciones, las clases no se pueden usar antes de definirlas. Las funciones en Javascript pasan por un proceso llamado hoisting por el cual las variables declaradas con var y las funciones son trasladadas al inicio del ámbito donde se encuentran y como consecuencia pueden ser llamadas antes de su declaración. En el caso de las clases esto no funciona así y si tratamos de usar una clase antes de definirla obtendremos un error.

## 6.3.2. HERENCIA

Aunque en ES5 podemos usar la cadena de prototipos para realizar una herencia, con la sintaxis de clases de ES6 esto es mucho más simple y claro. Para esto debemos usar extends en la declaración de la clase:

```
class Person {
  constructor(firstname, lastname, birthday) {
    this.firstname = firstname;
    this.lastname = lastname;
    this.birthday = new Date(birthday);
  }
   getAge() {
    const today = new Date();
    let age = today.getFullYear() - this.birthday.getFullYear();
    return age;
  }
}
class Employee extends Person {
  constructor(firstname, lastname, birthday, position) {
    super(firstname, lastname, birthday);
    this.position = position;
  }
}
```

```
// Creamos un objeto
var p = new Employee('Paul', 'Almunia', '07-08-1966 ', 'Architect');
// Comprobamos las propiedades
console.log(p.firstname, p.lastname, p.position);
console.log(p.getAge());
```

Para que un constructor o método pueda llamar a miembros de la clase principal, debe usar **super**, que es una referencia a la clase superior. En el caso del constructor se llamará simplemente con los parámetros. En el caso de querer llamar a métodos o propiedades concretas usaremos super seguido del nombre del método o propiedad de la clase padre (ejemplo: super.sonido() se podría llamar para heredar todo lo anterior y añadirle algo nuevo).

## 6.4. NOTACIÓN JSON

JSON es el acrónimo de ***JavaScript Objects Notation*** **(Notación de Objetos de JavaScript) que se creó en 2001 por parte de Douglas Crockford** con la idea de aprovechar la forma que posee JavaScript de crear objetos estáticos para crear un formato documental que sea más cómodo y eficiente para los programadores que los lenguajes de marcado como XML.

Aunque inicialmente se creó para ser usado desde JavaScript, actualmente se considera un formato documental independiente de cualquier lenguaje. La información almacenada en formato JSON se puede manipular desde casi cualquier lenguaje de programación actual.

El formato JSON es muy parecido a la forma de crear objetos de JavaScript, se basa en indicar propiedades y valores separados por dos puntos. Los valores se indican igual que en JavaScript (textos entrecomillados, los números tal cual y usando el punto como decimal, etc.) Se permite el uso de arrays, anidar objetos, etc. Pero hay diferencias que hay que reseñar:

- JSON solo admite definir propiedades, no se pueden indicar métodos en el formato JSON.
- En JSON el nombre de las propiedades tiene que ir entre comillas dobles (no se admiten el resto de las comillas

Ejemplo:

```
{
  "título": "Manual de UFOlogía",
  "n-serie": "187C2",
  "autores": ["Pedro Martínez", "Ana León"],
  "editorial": {
    "nombre": "Inexistente S.A.",
    "país": "España"
  },
  "edición": 2,
  "ensayo": true
}
```

Manipular datos en JSON es muy importante en la creación actual de aplicaciones web, por ello, JavaScript aporta un objeto global llamado **JSON** que permite manipular los datos en este formato.

Este objeto posee dos métodos:

- **stringify.** Sirve para convertir un objeto JavaScript a un string que contiene el formato JSON equivalente.
- **parse.** Recibe un texto en formato JSON y evalúa su corrección. Si es correcto, retorna el objeto equivalente y si no, devuelve una excepción de tipo **SyntaxError.**

Ejemplo de stringify:

```
const musico1 = {
  nombre: 'Bob',
  apellido: 'Dylan',
  fecha_nacimiento: { dia: 24, mes: 5, año: 1941 },
  discos: ['Highway 61 Revisited', 'Blonde on Blonde', 'Self Portrait'],
};
console.log(JSON.stringify(musico1));
```

El resultado:

**{"nombre":"Bob","apellido":"Dylan","fecha_nacimiento":{"dia":24,"mes":5,"año":1941},"discos":["High way 61 Revisited","Blonde on Blonde","Self Portrait"]}**

El método **parse** haría lo contrario, a partir del texto crea el objeto. Más adelante veremos las enormes funcionalidades que otorga JSON.